

Documento de Migração Arquitetural – Sistema de Controle Financeiro

Objetivo

Migrar o sistema atual para uma arquitetura moderna, escalável e de fácil manutenção, baseada em:

- Frontend: React + Material UI
 - Backend: Python + FastAPI
 - ORM: SQLAlchemy
 - Banco de Dados: PostgreSQL
 - Autenticação: JWT
 - Containerização: Docker
 - Relatórios: PDF e Excel
 - Controle de versão do banco: Alembic
-

Arquitetura Proposta

Frontend

Tecnologias

- React
- React Router
- React Query
- Material UI
- Axios
- Recharts

Estrutura

frontend/

├── src/

| ├── pages/

| ├── components/

| ├── layouts/

| ├── hooks/

| └── services/

| |—— contexts/

| |—— routes/

| |—— utils/

Backend

Tecnologias

- Python 3.13+
- FastAPI
- SQLAlchemy
- Pydantic
- Alembic
- JWT
- PostgreSQL Driver

Estrutura

backend/

|—— app/

| |—— api/

| |—— models/

| |—— schemas/

| |—— services/

| |—— repositories/

| |—— core/

| |—— database/

| |—— main.py

Padrão Arquitetural

Models

Responsáveis pelo mapeamento das tabelas.

Exemplo:

- Usuario
 - Conta
 - Categoria
 - Movimentacao
 - Cartao
 - Fatura
-

Schemas

Responsáveis pela validação dos dados de entrada e saída.

Exemplo:

- UsuarioCreate
 - UsuarioResponse
 - MovimentacaoCreate
 - MovimentacaoUpdate
-

Repositories

Responsáveis pelo acesso ao banco de dados.

Nenhuma regra de negócio deve existir nesta camada.

Services

Responsáveis pelas regras de negócio.

Exemplos:

- cálculo de saldo
 - fechamento de fatura
 - geração de fluxo de caixa
 - geração de indicadores
-

API

Responsável apenas pela exposição dos endpoints REST.

Banco de Dados

Tabela Usuarios

Campos:

- id
 - nome
 - email
 - senha_hash
 - ativo
 - criado_em
-

Tabela Contas

Campos:

- id
 - usuario_id
 - nome
 - banco
 - saldo_inicial
 - ativa
-

Tabela Categorias

Campos:

- id
- nome
- tipo

Valores possíveis:

- RECEITA
 - DESPESA
-

Tabela Movimentacoes

Campos:

- id
- usuario_id
- conta_id
- categoria_id

- tipo
- valor
- descricao
- data_movimento
- criado_em

Valores possíveis para tipo:

- RECEITA
 - DESPESA
 - TRANSFERENCIA
 - AJUSTE
-

Tabela Cartoes

Campos:

- id
 - usuario_id
 - nome
 - limite
 - fechamento
 - vencimento
-

Tabela ComprasCartao

Campos:

- id
 - cartao_id
 - descricao
 - valor_total
 - parcelas
 - parcela_atual
-

Tabela Faturas

Campos:

- id
 - cartao_id
 - competencia
 - valor_total
 - vencimento
 - status
-

Segurança

Autenticação

JWT Bearer Token

Fluxo:

1. Usuário realiza login.
 2. Sistema valida credenciais.
 3. Sistema gera JWT.
 4. Frontend utiliza token nas requisições.
-

Senhas

Utilizar:

- Argon2 (preferencial) ou
- Bcrypt

Nunca armazenar senhas em texto puro.

Dashboard

Indicadores principais:

- Saldo Atual
 - Receitas do Mês
 - Despesas do Mês
 - Resultado do Mês
 - Fluxo de Caixa
 - Gastos por Categoria
 - Evolução Patrimonial
-

Relatórios

Excel

- Receitas por período
- Despesas por categoria
- Fluxo de caixa
- Movimentações

Biblioteca:

- openpyxl
-

PDF

- Demonstrativo financeiro
- Prestação de contas
- Fechamento mensal

Biblioteca:

- WeasyPrint
-

Docker

Containers:

- frontend
- backend
- postgres

Estrutura:

docker-compose.yml

services:

- frontend
 - backend
 - postgres
-

Controle de Migração

Utilizar Alembic.

Objetivos:

- Versionar alterações do banco
 - Permitir rollback
 - Garantir rastreabilidade
-

Melhorias Futuras

Redis

- Cache
 - Controle de sessão
 - Filas
-

Celery

- Geração de relatórios assíncronos
 - Importação OFX
 - Processamentos agendados
-

MinIO

Armazenamento de:

- PDFs
 - Extratos
 - Comprovantes
-

Critérios de Sucesso

1. Sistema totalmente containerizado.
2. Banco PostgreSQL versionado via Alembic.
3. Backend desacoplado em camadas.
4. Frontend React responsivo.
5. APIs documentadas via Swagger.
6. Autenticação JWT funcional.
7. Relatórios PDF e Excel operacionais.
8. Estrutura preparada para escalabilidade futura.